

第 22 章

K 均值聚类

当我们要预测的是一个离散值时，做的工作就是“分类”。例如，要预测一个孩子能否成为优秀的运动员，其实就是要将他分到“好苗子”（能成为优秀的运动员）或“普通孩子”（不能成为优秀运动员）的类别。当我们要预测的是一个连续值时，做的工作就是“回归”。例如，预测一个孩子将来成为运动员的指数，计算得到的是 0.99 或者 0.36 之类的数值。

机器学习模型还可以将训练集中的数据划分为若干个组，每个组被称为一个“簇(cluster)”。这些自动形成的簇，可能对应着不同的潜在概念，例如“篮球苗子”、“长跑苗子”。这种学习方式被称为“聚类(clustering)”，它的重要特点是在学习过程中不需要用标签对训练样本进行标注。也就是说，学习过程能够根据现有训练集自动完成分类（聚类）。

根据训练数据是否有标签，我们可以将学习划分为监督学习和无监督学习。前面介绍的 K 近邻、支持向量机都是监督学习，提供有标签的数据给算法学习，然后对数据分类。而聚类是无监督学习，事先并不知道分类标签是什么，直接对数据分类。

举一个简单的例子，有 100 粒豆子，如果已知其中 40 粒为绿豆，40 粒为大豆，根据上述标签，将剩下的 20 粒豆子划分为绿豆和大豆则是监督学习。针对上述问题可以使用 K 近邻算法，计算当前待分类豆子的大小，并找出距离其最近的 5 粒豆子的大小，判断这 5 粒豆子中哪种豆子最多，将当前豆子判定为数量最多的那一类豆子类别人别。

同样，有 100 粒豆子，我们仅仅知道这些豆子里有两个不同的品种，但并不知道到底是什么品种。此时，可以根据豆子的大小、颜色属性，或者根据大小和颜色的组合属性，将其划分为两个类型。在此过程中，我们没有使用已知标签，也同样完成了分类，此时的分类是一种无监督学习。

聚类是一种无监督学习，它能够将具有相似属性的对象划分到同一个集合（簇）中。聚类方法能够应用于所有对象，簇内的对象越相似，聚类算法的效果越好。

22.1 理论基础

本节首先用一个实例来介绍 K 均值聚类的基本原理，在此基础上介绍 K 均值聚类的基本步骤，最后介绍一个二维空间下的 K 均值聚类示例。

22.1.1 分豆子

假设有 6 粒豆子混在一起，我们可以在不知道这些豆子类别人别的情况下，将它们按照直径大



小划分为两类。

经过测量，以 mm（毫米）为单位，这些豆子的直径大小分别为 1、2、3、10、20、30。下面将它们标记为 A、B、C、D、E、F，并进行分类操作。

第 1 步：随机选取两粒参考豆子。例如，随机将直径为 1mm 的豆子 A 和直径为 2 mm 的豆子 B 作为分类参考豆子。

第 2 步：计算每粒豆子的直径距离豆子 A 和豆子 B 的距离。距离哪个豆子更近，就将新豆子划分在哪个豆子所在的组。使用直径作为距离计算依据时，计算结果如表 22-1 所示。

表 22-1 每粒豆子与参考豆子的距离以及分组

豆子编号	直径 (mm)	与豆子 A 的距离 (mm)	与豆子 B 的距离 (mm)	分在何组
C	3	$3-1=2$	$3-2=1$	B
D	10	$10-1=9$	$10-2=8$	B
E	20	$20-1=19$	$20-2=18$	B
F	30	$30-1=29$	$30-2=28$	B

在本步骤结束时，6 粒豆子被划分为以下两组。

- 第 1 组：只有豆子 A。
- 第 2 组：豆子 B、C、D、E、F，共 5 粒豆子。

第 3 步：分别计算第 1 组豆子和第 2 组豆子的直径平均值。然后，将各个豆子按照与直径平均值的距离大小分组。

- 计算第 1 组豆子的平均值 $AV1 = 1\text{mm}$ 。
- 计算第 2 组豆子的平均值 $AV2 = (2+3+10+20+30)/5 = 13\text{mm}$ 。

得到上述平均值以后，对所有的豆子再次分组：

- 将平均值 AV1 所在的组，标记为 AV1 组。
- 将平均值 AV2 所在的组，标记为 AV2 组。

计算各粒豆子距离平均值 AV1 和 AV2 的距离，并确定分组，如表 22-2 所示。

表 22-2 每粒豆子与平均值 AV1 和 AV2 的距离以及新分组

豆子编号	直径 (mm)	与平均值 AV1 的距离 (mm)	与平均值 AV2 的距离 (mm)	分在何组
A	1	$1-1=0$	$13-1=12$	AV1
B	2	$2-1=1$	$13-2=11$	AV1
C	3	$3-1=2$	$13-3=10$	AV1
D	10	$10-1=9$	$13-10=3$	AV2
E	20	$20-1=19$	$20-13=7$	AV2
F	30	$30-1=29$	$30-13=17$	AV2

距离平均值 AV1 更近的豆子，就被划分为 AV1 组；距离平均值 AV2 更近的豆子，就被划分为 AV2 组。现在，6 粒豆子的分组情况为：

- AV1 组：豆子 A、豆子 B、豆子 C。
- AV2 组：豆子 D、豆子 E、豆子 F。

第 4 步：重复第 3 步，直到分组稳定不再发生变化，即可认为分组完成。



在本例中，重新计算 AV1 组的平均值 AV41、AV2 组的平均值 AV42，依次计算每个豆子与平均值 AV41 和 AV42 的距离，并根据该距离重新划分分组。按照与第 3 步相同的方法，重新计算平均值并分组后，6 粒豆子的分组情况为：

- AV41 组：豆子 A、豆子 B、豆子 C。
- AV42 组：豆子 D、豆子 E、豆子 F。

与上一次的分组相比，并未发生变化，我们就认为分组完成了。我们将直径较小的那一组称为“小豆子”，直径较大的那一组称为“大豆子”。

当然，本例是比较极端的例子，数据很快就实现了收敛，在实际处理中可能需要进行多轮的迭代才能实现数据的收敛，分类不再发生变化。

22.1.2 K均值聚类的基本步骤

K 均值聚类是一种将输入数据划分为 k 个簇的简单的聚类算法，该算法不断提取当前分类的中心点（也称为质心或重心），并最终在分类稳定时完成聚类。从本质上说，K 均值聚类是一种迭代算法。

K 均值聚类算法的基本步骤如下：

1. 随机选取 k 个点作为分类的中心点。
2. 将每个数据点放到距离它最近的中心点所在的类中。
3. 重新计算各个分类的数据点的平均值，将该平均值作为新的分类中心点。
4. 重复步骤 2 和步骤 3，直到分类稳定。

在第 1 步中，可以是随机选取 k 个点作为分类的中心点，也可以是随机生成 k 个并不存在于原始数据中的数据点作为分类中心点。

在第 3 步中，提到的“距离最近”，说明要进行某种形式的距离计算。在具体实现时，可以根据需要采用不同形式的距离度量方法。当然，不同的计算方法会对算法的性能产生影响。

22.2 K 均值聚类模块

OpenCV 提供了函数 `cv2.kmeans()` 来实现 K 均值聚类。该函数的语法格式为：

```
retval, bestLabels, centers=cv2.kmeans(data, K, bestLabels, criteria, attempts, flags)
```

式中各个参数的含义为：

- **data**：输入的待处理数据集合，应该是 `np.float32` 类型，每个特征放在单独的一列中。
- **K**：要分出的簇的个数，即分类的数目，最常见的是 $K=2$ ，表示二分类。
- **bestLabels**：表示计算之后各个数据点的最终分类标签（索引）。实际调用时，参数 `bestLabels` 的值设置为 `None`。
- **criteria**：算法迭代的终止条件。当达到最大循环数目或者指定的精度阈值时，算法停止继续分类迭代计算。该参数由 3 个子参数构成，分别为 `type`、`max_iter` 和 `eps`。



`type` 表示终止的类型，可以是三种情况，分别为：

- `cv2.TERM_CRITERIA_EPS`：精度满足 `eps` 时，停止迭代。
- `cv2.TERM_CRITERIA_MAX_ITER`：迭代次数超过阈值 `max_iter` 时，停止迭代。
- `cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER`：上述两个条件中的任意一个满足时，停止迭代。
- `max_iter`：最大迭代次数。
- `eps`：精确度的阈值。
- `attempts`：在具体实现时，为了获得最佳分类效果，可能需要使用不同的初始分类值进行多次尝试。指定 `attempts` 的值，可以让算法使用不同的初始值进行多次（`attempts` 次）尝试。
- `flags`：表示选择初始中心点的方法，主要有以下 3 种。
 - `cv2.KMEANS_RANDOM_CENTERS`：随机选取中心点。
 - `cv2.KMEANS_PP_CENTERS`：基于中心化算法选取中心点。
 - `cv2.KMEANS_USE_INITIAL_LABELS`：使用用户输入的数据作为第一次分类中心点；如果算法需要尝试多次（`attempts` 值大于 1 时），后续尝试都是使用随机值或者半随机值作为第一次分类中心点。

返回值的含义为：

- `retval`：距离值（也称密度值或紧密度），返回每个点到相应中心点距离的平方和。
- `bestLabels`：各个数据点的最终分类标签（索引）。
- `centers`：每个分类的中心点数据。

22.3 简单示例

本节通过几个简单示例来说明如何使用 OpenCV 中提供的函数 `cv2.kmeans()` 实现 K 均值聚类。

【例 22.1】 随机生成一组数据，使用函数 `cv2.kmeans()` 对其分类。

为了方便理解，假设有两种豆子，其中一种是“xiaoMI”，另外一种是“daMI”。它们的直径不一样，xiaoMI 的直径在 [0, 50] 区间；daMI 的直径在 [200, 250] 区间。用随机数模拟两种豆子的直径，并使用函数 `cv2.kmeans()` 对它们分类。

根据题目要求，主要步骤如下：

1. 数据预处理

使用随机函数随机生成两组豆子的直径数据，并将它们转换为函数 `cv2.kmeans()` 可以处理的格式。

2. 设置参数

设置函数 `cv2.kmeans()` 的参数形式。将参数 `criteria` 的值设置为“`(cv2.TERM_CRITERIA_EPS`

+ cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)”，在达到一定次数或者满足一定精度时终止迭代。

3. 调用函数cv2.kmeans()

调用函数 cv2.kmeans()，获取返回值，用于后续步骤的操作。

4. 确定分类

根据函数 cv2.kmeans()返回的标签（“0”和“1”），将原始数据分为两组。

5. 显示结果

绘制经过分类的数据及中心点，观察分类结果。

根据上述分析，编写代码如下：

```
import numpy as np
import cv2
from matplotlib import pyplot as plt
# 随机生成两组数组
# 生成 60 个值在[0,50]内的 xiaoMI 直径数据
xiaoMI = np.random.randint(0,50,60)
# 生成 60 个值在[200,250]内的 daMI 直径数据
daMI = np.random.randint(200,250,60)
# 将 xiaoMI 和 daMI 组合为 MI
MI = np.hstack((xiaoMI,daMI))
# 使用 reshape 函数将其转换为(120,1)
MI = MI.reshape((120,1))
# 将 MI 转换为 float32 类型
MI = np.float32(MI)
# 调用 kmeans 模块
# 设置参数 criteria 的值
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
# 设置参数 flags 的值
flags = cv2.KMEANS_RANDOM_CENTERS
# 调用函数 kmeans
retval,bestLabels,centers = cv2.kmeans(MI,2,None,criteria,10,flags)
'''
# 打印返回值
print(retval)
print(bestLabels)
print(centers)
'''
# 获取分类结果
XM = MI[bestLabels==0]
DM = MI[bestLabels==1]
# 绘制分类结果
# 绘制原始数据
plt.plot(XM, 'ro')
plt.plot(DM, 'bo')
```



```
# 绘制中心点
plt.plot(centers[0], 'rx')
plt.plot(centers[1], 'bx')
plt.show()
```

运行上述程序，程序运行结果如图 22-1 所示。

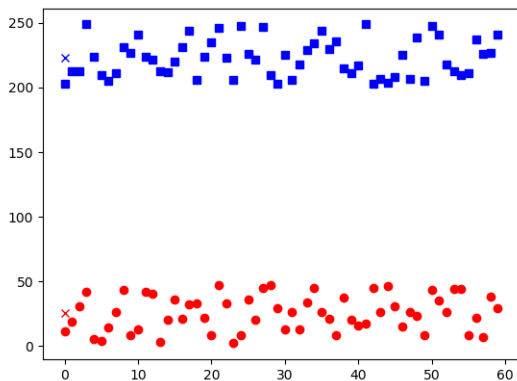


图 22-1 【例 22.1】程序的运行结果

在图 22-1 中，上面的小方块是标签为“0”的数据点，下方的圆点是标签为“1”的数据点。上方的“x”标记是标签为“0”的数据组的中心点，其值大概在 225 左右；下方的“x”是标签为“1”的数据组的中心点，其值大概在 25 左右。在需要时，可以通过 print 语句打印 centers[0]和 centers[1]获取两个中心点的值。

【例 22.2】有一堆米粒，按照长度和宽度对它们分类。

为了方便理解，假设米粒有两种，其中一种是 XM，另外一种 DM。它们的直径不一样，XM 的长和宽都在[0, 20]内，DM 的长和宽都在[40, 60]内。使用随机数模拟两种米粒的长度和宽度，并使用函数 cv2.kmeans()对它们分类。

根据题目要求，主要步骤如下：

- (1) 随机生成两组米粒的数据，并将它们转换为函数 cv2.kmeans()可以处理的形式。
- (2) 设置函数 cv2.kmeans()的参数形式。
- (3) 调用函数 cv2.kmeans()。
- (4) 根据函数 cv2.kmeans()的返回值，确定分类结果。
- (5) 绘制经过分类的数据及中心点，观察分类结果。

根据上述分析，编写代码如下：

```
import numpy as np
import cv2
from matplotlib import pyplot as plt
# 随机生成两组数值
# xiaomi 组, 长和宽都在[0, 20]内
xiaomi = np.random.randint(0, 20, (30, 2))
#dami 组, 长和宽的大小都在[40, 60]
```

```

dami = np.random.randint(40,60,(30,2))
# 组合数据
MI = np.vstack((xiaomi,dami))
# 转换为 float32 类型
MI = np.float32(MI)
# 调用 kmeans 模块
# 设置参数 criteria 值
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
# 调用 kmeans 函数
ret,label,center=cv2.kmeans(MI,2,None,criteria,10,cv2.KMEANS_RANDOM_CENTERS)
'''

#打印返回值
print(ret)
print(label)
print(center)
'''

# 根据 kmeans 的处理结果, 将数据分类, 分为 XM 和 DM 两大类
XM = MI[label.ravel()==0]
DM = MI[label.ravel()==1]
# 绘制分类结果数据及中心点
plt.scatter(XM[:,0],XM[:,1],c = 'g', marker = 's')
plt.scatter(DM[:,0],DM[:,1],c = 'r', marker = 'o')
plt.scatter(center[0,0],center[0,1],s = 200,c = 'b', marker = 'o')
plt.scatter(center[1,0],center[1,1],s = 200,c = 'b', marker = 's')
plt.xlabel('Height'),plt.ylabel('width')
plt.show()

```

运行上述程序, 结果如图 22-2 所示。

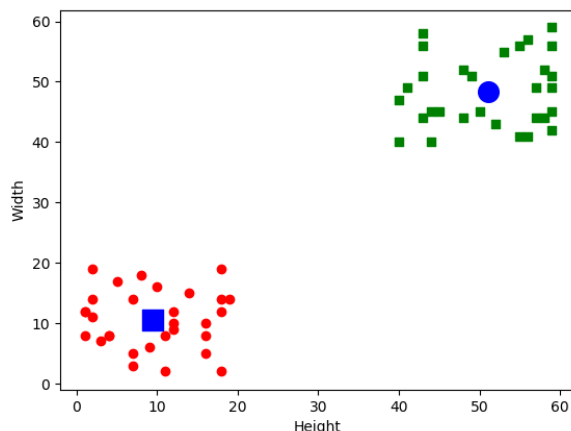


图 22-2 【例 22.2】程序的运行结果

在图 22-2 中, 右上方的方块是标签为“0”的数据点, 左下方的圆点是标签为“1”的数据点。右上方稍大的圆点是标签“0”的数据组的中心点; 左下方稍大的方块是标签为“1”的数据组的中心点。

在程序中，“#打印返回值”下面 3 行的打印语句被注释掉了。在需要时，可以去掉注释，通过 `print` 语句打印对应的值。例如，语句“`print(center)`”可以打印 `center` 所表示的两个中心点的值。

【例 22.3】使用函数 `cv2.kmeans()` 将灰度图像处理为只有两个灰度级的二值图像。

根据题目要求，需要对灰度图像内的色彩进行分类，将所有的像素点划分为两类。然后，用这两类的中心点像素值替代原有像素值，满足题目的要求。

根据分析，主要步骤如下：

1. 图像预处理

读取图像，并将图像转换为函数 `cv2.kmeans()` 可以处理的形式。

在读取图像时，如果是 3 个通道的 RGB 图像，需要将图像的 RGB 值处理为一个单独的特征值。具体实现时，用函数 `cv2.reshape()` 完成对图像特征值的调整。

为了满足函数 `cv2.kmeans()` 的要求，需要将图像的数据类型转换为 `numpy.float32` 类型。

2. 设置函数 `cv2.kmeans()` 的参数形式

设置参数 `criteria` 的值为“`(cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)`”，让函数 `cv2.kmeans()` 在达到一定精度或者达到一定迭代次数时，即停止迭代。

设置参数 `K` 的值为 2，将所有像素划分为两类。

3. 调用函数 `cv2.kmeans()`

调用函数 `cv2.kmeans()`，得到距离值、分类中心点和分类标签，用于后续操作。

4. 值替换

将像素点的值替换为当前分类的中心点的像素值。

5. 显示变换前后的图像

分别显示原始图像和二值化图像。

根据上述分析，编写代码如下：

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
# 读取待处理图像
img = cv2.imread('lena.bmp')
# 使用 reshape 将一个像素点的 RGB 值作为一个单元处理
data = img.reshape((-1,3))
# 转换为 kmeans 可以处理的类型
data = np.float32(data)
# 调用 kmeans 模块
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
K = 2
```

```

ret, label, center=cv2.kmeans(data,K,None,criteria,10,cv2.KMEANS_RANDOM_CENTER
S)
# 转换为 uint8 数据类型, 将每个像素点都赋值为当前分类的中心点像素值
# 将 center 的值转换为 uint8
center = np.uint8(center)
# 使用 center 内的值替换原像素点的值
res1 = center[label.flatten()]
# 使用 reshape 调整替换后的图像
res2 = res1.reshape((img.shape))
# 显示处理结果
plt.subplot(121)
plt.imshow(img)
plt.axis('off')
plt.subplot(122)
plt.imshow(res2)
plt.axis('off')

```

运行上述程序, 结果如图 22-3 所示。其中, 左图是原始图像, 右图是二值化图像。调整程序中的 K 值, 就能改变图像的显示结果。例如, K=8, 则可以让图像显示 8 个灰度级。



图 22-3 【例 22.3】程序的运行结果